S.No: 1	Exp. Name: ***Largest of Three Numbers***	Date: 2026-09-22

Aim:

Write a Python program that prompts the user to enter three integers. Print the largest of the three integers.

Input Format:

- The program will prompt the user to enter three integers, one per line.

Output Format:

- The output will display the largest integer among the three integers.

Source Code:

largestNumber.py

```
#write your code here...
a = int(input())
b = int(input())
c = int(input())
print(max(a,b,c))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
6
7
7

Test Case - 2
User Output
14
26
19
26

ID: 25ME1A5483

Ramachandra College of Engineering

S.No: 2	Exp. Name: ***Prime Numbers in an Interval***	**Date: 2026-09-22**

Aim:

Write a Python program to print all prime numbers in a given interval. The program should use the `break` statement to optimize the check for prime numbers.

Input Format:

- The first input line reads an integer representing the starting number of the interval.
- The second input line reads an integer representing the ending number of the interval.

Output Format:

- Print all prime numbers within the given interval(both are inclusive), each printed on a new line.

Source Code:

Prime_Numbers_in_an_Interval.py

```
# Write your code here..
start = int(input())
end = int(input())
for n in range(start,end + 1):
        if n > 1:
                for i in range(2, int(n**0.5) + 1):
                        if n % i == 0:
                                break
                else:
                        print(n)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1
10
2
3
5

ID: 25ME1A5483

Ramachandra College of Engineering

7

Test Case - 2
User Output
2
15
2
3
5
7
11
13

S.No: 3	Exp. Name: ***Swapping of Two Numbers***	Date: 2026-09-22

Aim:

Write a Python program to swap two integers without using a third variable.

Input Format:

- The first line contains an integer, $num1$, representing the first number.
- The second line contains an integer, $num2$, representing the second number.

Output Format:

- Print the numbers before swapping on the first line, separated by a space.
- Print the numbers after swapping on the second line, separated by a space.

Note: Refer to the visible test cases for better understanding.

Source Code:

swapnum.py

```
# write your code here
num1 = int(input("num1: "))
num2 = int(input("num2: "))
print("numbers before swapping : {} {}".format(num1, num2))
num1,num2=num2,num1
print("numbers after swapping : {} {}".format(num1, num2))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
num1:
10
num2:
20
numbers before swapping : 10 20
numbers after swapping : 20 10

Test Case - 2

User Output
num1:
0
num2:
0
numbers before swapping : 0 0
numbers after swapping : 0 0

Test Case - 3
User Output
num1:
-123
num2:
124
numbers before swapping : -123 124
numbers after swapping : 124 -123

S.No: 4	Exp. Name: ***Arithmetic Operations (Addition, Subtraction, Multiplication, Division)***	**Date: 2026-09-22**

Aim:

Write a program to read two positive integer inputs from the user and perform the following arithmetic operations: addition, subtraction, multiplication, and division, and print the result.

Input Format:

- The first two lines represent the input of two integers $num1$ and $num2$.

Output Format:

- The first line should print the sum of the provided inputs.
- The second line should print the subtraction of the provided inputs.
- The third line should print the multiplication of the provided inputs.
- The fourth line should print the division of the provided inputs.

Sample Test Case:

Input:

```
num1: 40
num2: 20
```

Output:

```
Addition of 40 and 20 = 60
Subtraction of 40 and 20 = 20
Multiplication of 40 and 20 = 800
Division of 40 and 20 = 2.0
```

Note:

- Assume the value of $num2$ is always greater than 0 (zero).

Source Code:

Arithexample2.py

```
# write the code...
x = int(input("num1: "))
y = int(input("num2: "))
print("Addition of {} and {} = {}".format(x, y, x+y))
print("Subtraction of {} and {} = {}".format(x, y, x-y))
print("Multiplication of {} and {} = {}".format(x, y, x*y))
print("Division of {} and {} = {}".format(x, y, x/y))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
num1:
40
num2:
20
Addition of 40 and 20 = 60
Subtraction of 40 and 20 = 20
Multiplication of 40 and 20 = 800
Division of 40 and 20 = 2.0

Test Case - 2
User Output
num1:
4
num2:
6
Addition of 4 and 6 = 10
Subtraction of 4 and 6 = -2
Multiplication of 4 and 6 = 24
Division of 4 and 6 = 0.6666666666666666

S.No: 5	Exp. Name: ***Comparison Operators***	Date: 2026-09-22

Aim:

Write a Python program that demonstrates the usage of comparison operators.
The program should:

- Read two integers from the user.
- Perform each of the six standard comparison operations.
- Print the result of each comparison in a descriptive format.

Input Format:

- First line: An integer representing $num1$.
- Second line: An integer representing $num2$.

Output Format:

- The output line contains boolean values which represent the results of various comparison operations `==`, `!=`, `<`, `<=`, `>`, `>=`, and the format should be like below:

```
<num1> <comparison operator> <num2> : <boolean result>
```

Source Code:

operators.py

```
#Write your code here...
num1 = int(input())
num2 = int(input())
print("{} == {} : {}".format(num1,num2,num1==num2))
print("{} != {} : {}".format(num1,num2,num1!=num2))
print("{} < {} : {}".format(num1,num2,num1<num2))
print("{} <= {} : {}".format(num1,num2,num1<=num2))
print("{} > {} : {}".format(num1,num2,num1>num2))
print("{} >= {} : {}".format(num1,num2,num1>=num2))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
4
4 == 4 : True

4 != 4 : False
4 < 4 : False
4 <= 4 : True
4 > 4 : False
4 >= 4 : True

Test Case - 2
User Output
3
2
3 == 2 : False
3 != 2 : True
3 < 2 : False
3 <= 2 : False
3 > 2 : True
3 >= 2 : True

Test Case - 3
User Output
5
5
5 == 5 : True
5 != 5 : False
5 < 5 : False
5 <= 5 : True
5 > 5 : False
5 >= 5 : True

S.No: 6	Exp. Name: ***Assignment Operations***	Date: 2026-09-22

Aim:

Write a Python program that takes two integers a and b as input from the user. Perform the following assignment operations on the given values in the exact order:

1. $a += b$
2. $a -= b$
3. $a *= b$
4. $a /= b$
5. $a **= b$
6. $a //= b$
7. $a \%= b$
8. $a = b$

After each operation, print the updated values of a and b in the format shown below.
Note: The value of a gets updated after each operation, and the updated a is used for the next operation.

Input Format:

- Two integers, a and b, each entered on separate lines.

Output Format:

- For each assignment operation, print the result in the following format:

```
a+=b: a=<value_of_a> b=<value_of_b>
a-=b: a=<value_of_a> b=<value_of_b>
a*=b: a=<value_of_a> b=<value_of_b>
a/=b: a=<value_of_a> b=<value_of_b>
a**=b: a=<value_of_a> b=<value_of_b>
a//=b: a=<value_of_a> b=<value_of_b>
a%=b: a=<value_of_a> b=<value_of_b>
a=b a=<value_of_a> b=<value_of_b>
```

Note : Your input and output layout must match the visible sample test case

Source Code:

assignment.py

```
# Write your code here
a = int(input())
b = int(input())
a += b
print("a+=b: a= {} b= {}".format(a,b))
a -= b
print("a-=b: a= {} b= {}".format(a,b))
a *= b
print("a*=b: a= {} b= {}".format(a,b))
a /= b
print("a/=b: a= {} b= {}".format(a,b))
a **= b
print("a**=b: a= {} b= {}".format(a,b))
a //= b
print("a//=b: a= {} b= {}".format(a,b))
a %= b
print("a%=b: a= {} b= {}".format(a,b))
a = b
print("a=b a= {} b= {}".format(a,b))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
2
3
a+=b: a= 5 b= 3
a-=b: a= 2 b= 3
a*=b: a= 6 b= 3
a/=b: a= 2.0 b= 3
a**=b: a= 8.0 b= 3
a//=b: a= 2.0 b= 3
a%=b: a= 2.0 b= 3
a=b a= 3 b= 3

S.No: 7	Exp. Name: ***Logical Operators***	Date: 2026-09-22

Aim:

Write a Python program that reads two integers and performs logical operations using the `and`, `or`, and `not` operators.

Input Format:

- The first line contains an integer $num1$.
- The second line contains an integer $num2$.

Output Format:

- Print the results of the logical operations in the following format:

```
<num1> and <num2>: <result>
<num1> or <num2>: <result>
not <num1>: <result>
not <num2>: <result>
```

Note: Refer to the visible test cases for better understanding.

Source Code:

logicoperator.py

```
# Write your code here...
first_number = int(input())
second_number = int(input())
print(f"{first_number} and {second_number}: {first_number and
second_number}")
print(f"{first_number} or {second_number}: {first_number or
second_number}")
print(f"not {first_number}: {not first_number}")
print(f"not {second_number}: {not second_number}")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
3
4
3 and 4: 4
3 or 4: 3

not 3: False
not 4: False

Test Case - 2
User Output
0
1
0 and 1: 0
0 or 1: 1
not 0: True
not 1: False

Test Case - 3
User Output
0
0
0 and 0: 0
0 or 0: 0
not 0: True
not 0: True

S.No: 8	Exp. Name: ***Shift operators***	Date: 2026-09-22

Aim:

Write a Python program that reads two integers and demonstrates the use of shift operators by performing left and right bitwise shift operations.

Input Format:

- The first line contains an integer a.
- The second line contains an integer b representing the number of positions to shift.

Output Format:

- Print the result of left-shifting a by b positions in the following format:

```
a << b = <result>
```

- Print the result of right-shifting a by b positions in the following format:

```
a >> b = <result>
```

Source Code:

shiftoperator.py

```
#write your code here
a = int(input())
b = int(input())
print(f"a << b = {a << b}")
print(f"a >> b = {a >> b}")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
5
7
a << b = 640
a >> b = 0

Test Case - 2
User Output

ID: 25ME1A5483

Ramachandra College of Engineering

2
5
a << b = 64
a >> b = 0

Test Case - 3
User Output
3
6
a << b = 192
a >> b = 0

S.No: 9	Exp. Name: ***Ternary Operator***	Date: 2026-09-22

Aim:

Write a python program to find the maximum value of given two numbers using ternary operator.

Input Format:

- First line should prompt the user to enter first integer
- Second line should prompt the user to enter second integer

Output Format:

- Output should print the maximum value using ternary operator

Source Code:

ternary.py

```
# Write your code here...
x = int(input())
y = int(input())
max_value = x if x>y else y
print(max_value)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
91
123
123

Test Case - 2
User Output
15
-23
15

S.No: 10	Exp. Name: ***Membership Operator***	Date: 2026-09-22

Aim:

Write a Python program to demonstrate the membership operator.

A predefined list of integers is already provided to you.

Input Format:

- A single line of input, n, representing an integer.

Output Format:

- Print 'True' or 'False' based on whether n is present in the list using the `in` operator.
- Print 'True' or 'False' based on whether n is not present in the list using the not `in` operator.

Note: This question does not have partial weightages for test cases. All the test cases must be passed.

Source Code:

membership.py

```
num_list = [1, 2, 3, 4, 5, 15, 25, 89, 54, 123, 489]
# Write your code here...
n = int(input())
print(n in num_list)
print(n not in num_list)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
21
False
True

Test Case - 2

ID: 25ME1A5483
2025-2029-AIDS-B
Ramachandra College of Engineering

User Output
-3
False
True

Test Case - 3
User Output
5
True
False

Test Case - 4
User Output
54
True
False

S.No: 11	Exp. Name: ***Writing using Identity "is"***	Date: 2026-09-22

Aim:

Take two integers `x` and `y` as input from the console using `input()` function. Using the identity operator `is` check if `x` and `y` are the same objects or not, print to the console, the result of the two input integers as shown in the example.

Sample Input and Output1:

```
Enter an integer value: 35
Enter an integer value: 35
35 is 35 True
```

Sample Input and Output2:

```
Enter an integer value: -19
Enter an integer value: -19
-19 is -19 False
```

Source Code:

IdenopExample2.py

```
# Enter both integers with message "Enter an integer value: "
# Make use of identity operator as required in the problem statement
x = int(input("Enter an integer value: "))
y = int(input("Enter an integer value: "))
result = x is y
print(f"{x} is {y} {result}")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter an integer value:
35
Enter an integer value:
35
35 is 35 True

Test Case - 2
User Output
Enter an integer value:
-19
Enter an integer value:
-19
-19 is -19 False

S.No: 12	Exp. Name: ***Writing an example using identity operator "is not"***	**Date: 2026-09-22**

Aim:

Take two integers `a` and `b` as input from the console using `input()` function. Using the identity operator `is not` check if `a` and `b` are same objects or not, print to the console, the result of the two input integers as shown in the example.

Sample Input and Output 1:

```
Enter an integer value: -18
Enter an integer value: -18
-18 is not -18 True
```

Sample Input and Output 2:

```
Enter an integer value: 156
Enter an integer value: 156
156 is not 156 False
```

Source Code:

IdenopExample3.py

```
# Input integers a and b with message "Enter an integer value: "
# Use identity operator is not as required in the problem statement
a = int(input("Enter an integer value: "))
b = int(input("Enter an integer value: "))
result = a is not b
print(f"{a} is not {b} {result}")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter an integer value:
-18
Enter an integer value:
-18

ID: 25ME1A5483

Ramachandra College of Engineering

-18 is not -18 True

Test Case - 2
User Output
Enter an integer value:
156
Enter an integer value:
156
156 is not 156 False

S.No: 13	Exp. Name: ***Addition and Multiplication of Two Complex Numbers***	**Date: 2026-09-22**

Aim:

Write a program that takes two complex numbers as input in the form $a + bi$ and $c + di$, and performs the following:

- Adds the two complex numbers.
- Multiplies the two complex numbers.

Input Format:

- The first line contains two integers a and b, representing the real and imaginary parts of the first complex number.
- The second line contains two integers c and d, representing the real and imaginary parts of the second complex number.

Output Format:

- Print the sum of the complex numbers in the form $x + yi$.
- Print the product of the complex numbers in the form $x + yi$.

Source Code:

ComplexNumOps.py

```
# Type Content here...
a, b = map(int, input().split())
c, d = map(int, input().split())
z1 = complex(a, b)
z2 = complex(c, d)
sum_result = z1+z2
product_result = z1*z2
sum_real = int(sum_result.real)
sum_imag = int(sum_result.imag)
prod_real = int(product_result.real)
prod_imag = int(product_result.imag)
print(f"{sum_real} + {sum_imag}i")
print(f"{prod_real} + {prod_imag}i")
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
-6 5

9 8
3 + 13i
-94 + -3i

Test Case - 2
User Output
0 3
0 4
0 + 7i
-12 + 0i

S.No: 14	Exp. Name: ***Multiplication Table***	Date: 2026-09-22

Aim:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

`<number> x <multiplier> = <result>`

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Source Code:

multiplicationTable.py

```
#write your code here...
n = int(input())
for i in range(1,11):
        print(n,"x",i,"=",n*i)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
8
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64

8 x 9 = 72
8 x 10 = 80

Test Case - 2
User Output
5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

S.No: 15	Exp. Name: ***Function with Multiple Return Values***	**Date: 2026-09-22**

Aim:

Write a Python program that defines a function **calculate_stats** to calculate multiple values from a given list of numbers. The function should return three values: the sum, the mean, and the maximum of the numbers.

Input Format:

- A single line containing space-separated integers representing the elements of the list

Output Format:

- The first line prints the sum of all the integers.
- The second line prints the mean (average) of the integers, formatted to two decimal places.
- The third line prints the maximum value among the integers.

Constraints:

- $0 < length\ of\ the\ list <= 100$

Source Code:

multiplefun.py

```
# write your code here
def calculate_stats(numbers):
        total = sum(numbers)
        mean = total/len(numbers)
        maximum = max(numbers)
        return total, mean, maximum
numbers = list(map(int, input().split()))
total, mean, maximum = calculate_stats(numbers)
print(total)
print(f"{mean:.2f}")
print(maximum)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
-23 -45 -56 -90 -12 -87

-313
-52.17
-12

Test Case - 2
User Output
12 23 34 45 56 67 78 89
404
50.50
89

S.No: 16	Exp. Name: ***Default Arguments Function***	Date: 2026-09-22

Aim:

Write a program to define a function **greet()** with default arguments. This function should accept three parameters: **name, age,** and **country**. If the user enters -1 for any input, the corresponding default value must be used. The default values are:

- **name** = "Dude"
- **age** = 0
- **country** = "Space"

The program should read the three inputs from the user. If the user enters **-1** for any input, the corresponding default value must be used.

Input Format:

- First line prompts the user to enter the *name* by displaying: "-1 for default: "
  - If -1 is entered, the default value "Dude" is used.
- Second line prompts the user to enter the *age* by displaying: "-1 for default: "
  - If -1 is entered, the default value "0" is used.
- Third line of input should prompt the user to enter the *country* by displaying: "-1 for default: "
  - If -1 is entered, the default value "Space" is used.

Output Format:

- Print the following line using the provided or default values:

```
Hello, {name}! You are {age} years old and you are from {country}
```

Source Code:

default.py

```
# Write your code here...
def greet(name = "Dude", age = 0, country = "Space"):
        print(f"Hello, {name}! You are {age} years old and you are from
{country}")
def get_user_input():
        name = input("-1 for default: ")
        age = int(input("-1 for default: "))
        country = input("-1 for default: ")
        if name == "-1":
                name = "Dude"
        if age == -1:
                age = 0
        if country == "-1":
                country = "Space"
        return name, age, country
name, age, country = get_user_input()
greet(name, age, country)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
-1 for default:
-1
-1 for default:
-1
-1 for default:
-1
Hello, Dude! You are 0 years old and you are from Space

Test Case - 2
User Output
-1 for default:
Ram
-1 for default:
-1
-1 for default:

-1
Hello, Ram! You are 0 years old and you are from Space

Test Case - 3
User Output
-1 for default:
Vikas
-1 for default:
26
-1 for default:
America
Hello, Vikas! You are 26 years old and you are from America

S.No: 17	Exp. Name: ***Length of the String***	Date: 2026-09-22

Aim:

Write a Python program to find the length of a string without using any library functions

Input Format:

- A single line containing a string s.

Output Format:

- A single integer representing the length of the input string.

Source Code:

length.py

```
# Write your code here...
def string_length(s):
        count = 0
        for _ in s:
                count += 1
        return count
input_string = input()
length = string_length(input_string)
print(length)
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Code Tantra
11

Test Case - 2
User Output
Hello@World!!
13

ID: 25ME1A5483

Ramachandra College of Engineering

Test Case - 3
User Output
12345678
8

S.No: 18	Exp. Name: ***Substring or Not***	Date: 2026-09-22

Aim:

You are given two strings. You need to determine whether either string is a substring of the other.

Input Format:

- First line of input contains a string $s1$.
- Second line of input contains a string $s2$.

Output Format:

- "Exists" if either $s1$ is a substring of $s2$, or $s2$ is a substring of $s1$. Otherwise, return "Does not exist".

Source Code:

subString.py

```
def is_substring(s1, s2):
        if s1 in s2 or s2 in s1:
                return "Exists"
        else:
                return "Does not exist"

s1 = input()
s2 = input()
print(is_substring(s1, s2))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Hello
Hello World
Exists

Test Case - 2
User Output

Collections
Occured Collision
Does not exist

S.No: 19	Exp. Name: ***List Operations***	Date: 2026-09-22

Aim:

Write a program to perform the following operations on a list.

1. Addition (add an element to the list)
2. Insertion (insert an element at a specified index)
3. Slicing (extract a sublist using start and end indexes)

Input Format:

- The first line should take space-separated integers to form the initial list.
- The second line of input should take two space-separated integers, representing the index and value for the insertion.
- The third line should take two space-separated integers, representing the start index and end index for slicing.

Output Format:

- First line should display the initial list.
- Second line should display the list after insertion.
- Third line should display the list after slicing.

Sample Input:

```
1 2 3 4 5
0 5
0 3
```

Sample Output:

```
[1, 2, 3, 4, 5]
[5, 1, 2, 3, 4, 5]
[5, 1, 2]
```

Note:

- In Python, `append()` adds an element to the end of a list.
- To insert an element at a specific position, use the `insert()` method.

Source Code:

listOp.py

```
# Type Content here...
numbers = list(map(int,input().split()))
index, value = map(int, input().split())
start, end = map(int,input().split())
print(numbers)
numbers.insert(index,value)
print(numbers)
print(numbers[start:end])
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1 2 3 4 5
0 5
0 3
[1, 2, 3, 4, 5]
[5, 1, 2, 3, 4, 5]
[5, 1, 2]

Test Case - 2
User Output
12 52 65 41 23 89 654
7 485
0 8
[12, 52, 65, 41, 23, 89, 654]
[12, 52, 65, 41, 23, 89, 654, 485]
[12, 52, 65, 41, 23, 89, 654, 485]

S.No: 20	Exp. Name: ***Program using Built in List Functions***	Date: 2026-09-22

Aim:

Write a Python program that performs the following operations on a list of integers provided by the user:

1. Calculate the number of elements in the list.
2. Find the maximum value in the list.
3. Find the minimum value in the list.
4. Compute the sum of all elements in the list.
5. Produce a sorted list of the elements in ascending order.

Input Format:

- The program should read a single line of input containing integers separated by spaces.

Output Format:

- First Line : Print the number of elements in the list.
- Second Line :Print the maximum value in the list.
- Third Line :Print the minimum value in the list.
- Fourth Line :Print the sum of all elements in the list.
- Fifth Line :Print the sorted list in ascending order.

Source Code:

listBuiltinFunctions.py

```
# Type Content here...
numbers = list(map(int,input().split()))
print(len(numbers))
print(max(numbers))
print(min(numbers))
print(sum(numbers))
print(sorted(numbers))
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1 4 2 5
4

5
1
12
[1, 2, 4, 5]

Test Case - 2
User Output
10 9 8 7 6
5
10
6
40
[6, 7, 8, 9, 10]